Iteration E1: Creating a Smarter Cart

Associating a count with each product in our cart is going to require us to modify the `line_items` table. We've used migrations before; for example, we used a migration in*Applying the Migration*, to update the schema of the database. While that was as part of creating the initial scaffolding for a model, the basic approach is the same:

```
depot> bin/rails generate migration add_quantity_to_line_items
quantity:integer
```

Rails can tell from the name of the migration that you're adding columns to the `line_items` table and can pick up the names and data types for each column from the last argument. The two patterns that Rails matches on are `AddXXXToTABLE` and `RemoveXXXFromTABLE`, where the value of `XXX` is ignored; what matters is the list of column names and types that appears after the migration name.

The only thing Rails can't tell is what a reasonable default is for this column. In many cases, a `null` value would do, but let's make it the value `1` for existing carts by modifying the migration before we apply it:

rails80/depot_g/db/migrate/20250420000005_add_quantity_to_line_items.rb

```
class AddQuantityToLineItems < ActiveRecord::Migration[8.0]
  def change
»   add_column :line_items, :quantity, :integer, default: 1
  end
end
```

Once it's complete, we run the migration:

```
depot> bin/rails db:migrate
```

Now we need a smart `add_product` method in our `Cart`, one that checks if our list of items already includes the product we're adding; if it does, it bumps